Lab 5: Static Code Analysis

AFTER: Fixed Code and Analysis Reports

Name: Vinodkumar L

SRN: PES1UG25AM817

File: premium_calculator.py

Tools: Pylint, Bandit, Flake8 (Python 3.12)

1. Results on the Fixed Code

Tool	Before	After
Pylint	5.73 / 10	10.00 / 10
Bandit	10 issues	0 issues
Flake8	19 warnings	0 warnings

Fixes were made in three commits, one per tool, plus a final commit for the documentation.

Commit	Message
1	Fix Bandit findings: safe XML parsing, TLS verification, tar extraction filter, secrets handling
2	Fix Pylint findings and logic bugs: snake_case, docstrings, Applicant dataclass, mutable default, age/renewal/instalment bugs (10.00/10)
3	Fix Flake8 findings: PEP 8 blank lines and line length (0 warnings)
4	Add issues.md and reflection.md

2. What Was Fixed

Security: unsafe tar extraction now uses the PEP 706 data filter; TLS certificate verification is on and only https URLs are accepted; partner XML is parsed with defusedxml; the service binds to 127.0.0.1; the underwriter token comes from an environment variable and is compared in constant time; policy numbers use secrets and never overwrite an existing quote; the log file no longer lives in /tmp.

Logic: the discount list is no longer shared or mutated, so identical customers get identical quotes (6000.0); smoker must be a real boolean; premiums are floored at 0 (the -800 quote is now 0.0); age accounts for the birthday (35); 29 February renews on 28 February; instalments are rounded to paise and months=0 is rejected.

Quality and style: docstrings, snake_case names, an Applicant dataclass (6 arguments down to 4), with-blocks for files, csv-safe log lines, a __main__ guard, and PEP 8 spacing and line length.

3. Reports on the Fixed Code

Pylint

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Bandit

```
[main]   INFO     profile include tests: None
[main]   INFO     profile exclude tests: None
```

```
[main]   INFO     cli include tests: None
[main]   INFO     cli exclude tests: None
[main]   INFO     running on Python 3.12.3
Run started:2026-10-26 08:00:42.962032+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 160
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 1

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

Flake8

```
(no output: no warnings reported)
```

4. Fixed Code

```
"""Insurance premium calculator.

Quotes health, motor and life premiums, works out an applicant's age, logs
quotes, imports policies from partner XML, downloads rate tables, extracts
customer claim archives, splits premiums into instalments and computes
renewal dates.
"""
import csv
import hmac
import logging
import os
import secrets
import ssl
import tarfile
import urllib.parse
import urllib.request
from dataclasses import dataclass
from datetime import date, datetime

from defusedxml import ElementTree as ET

# Global quote book
quotes = {}
# Bind to loopback by default; expose on other interfaces only via config.
HOST = os.environ.get("PREMIUM_HOST", "127.0.0.1")
# Secret comes from the environment, never from source control.
UNDERWRITER_TOKEN = os.environ.get("UNDERWRITER_TOKEN", "")
# Log lives in the working directory (override with PREMIUM_LOG_FILE), not in
# the world-writable /tmp where another user could pre-create or symlink it.
LOG_FILE = os.environ.get("PREMIUM_LOG_FILE", "premium_quotes.log")
BASE_RATES = {"health": 5000, "motor": 3000, "life": 8000}
SENIOR_AGE = 45
SENIOR_LOADING = 1.5
SMOKER_LOADING = 1.3
RATE_PER_RUPEE_INSURED = 0.002
CLAIM_LOADING = 1000



@dataclass(frozen=True)
class Applicant:
    """The risk factors about the person being quoted."""

    age: int
    smoker: bool
    claims: int = 0

    def __post_init__(self):
        # A truthy string such as "no" used to count as a smoker.
        if not isinstance(self.smoker, bool):
            raise TypeError("smoker must be True or False")
        if self.age < 0 or self.claims < 0:
            raise ValueError("age and claims cannot be negative")



def calculate_premium(policy_type, applicant, sum_insured, discounts=None):
    """Return the annual premium in rupees, rounded to 2 decimals.

    ``discounts`` is a list of rupee amounts to subtract. It is only read,
    never modified, so callers can safely reuse the same list. The premium
    never goes below zero.
    """
    if policy_type not in BASE_RATES:
        raise ValueError(f"Unknown policy type: {policy_type!r}")
    if sum_insured < 0:
        raise ValueError("sum_insured cannot be negative")
    discounts = discounts or []
    if any(d < 0 for d in discounts):
        raise ValueError("discounts cannot be negative")

    premium = BASE_RATES[policy_type] + sum_insured * RATE_PER_RUPEE_INSURED
    if applicant.age > SENIOR_AGE:
        premium *= SENIOR_LOADING
    if applicant.smoker:
        premium *= SMOKER_LOADING
    premium += applicant.claims * CLAIM_LOADING
    premium -= sum(discounts)
    return round(max(premium, 0.0), 2)
```

```
def age_from_dob(dob, today=None):
    """Return completed years of age for a DD-MM-YYYY date of birth."""
    born = datetime.strptime(dob, "%d-%m-%Y").date()
    today = today or date.today()
    if born > today:
        raise ValueError("Date of birth is in the future")
    had_birthday = (today.month, today.day) >= (born.month, born.day)
    return today.year - born.year - (0 if had_birthday else 1)



def policy_number():
    """Return a random six-digit policy number such as POL-123456."""
    return "POL-" + str(100000 + secrets.randbelow(900000))



def save_quote(name, premium):
    """Store a quote in the quote book and append it to the log file."""
    number = policy_number()
    while number in quotes:  # never overwrite an existing quote
        number = policy_number()
    quotes[number] = {"name": name, "premium": premium}
    with open(LOG_FILE, "a", encoding="utf-8", newline="") as log:
        # csv quotes names that contain commas or newlines.
        csv.writer(log).writerow([number, name, f"{premium:.2f}"])
    return number



def import_policies(xml_text):
    """Parse partner XML into (type, age, sum) tuples, skipping bad rows."""
    root = ET.fromstring(xml_text)
    result = []
    for policy in root.findall("policy"):
        try:
            policy_type = policy.get("type")
            age = int(policy.get("age"))
            sum_insured = int(policy.get("sum"))
            if policy_type not in BASE_RATES or age < 0 or sum_insured < 0:
                raise ValueError("value out of range")
            result.append((policy_type, age, sum_insured))
        except (TypeError, ValueError):
            logging.warning(
                "Skipping malformed policy element: %s", policy.attrib
            )
    return result



def fetch_rates(url):
    """Download a rate table over verified HTTPS and return the bytes."""
    if urllib.parse.urlparse(url).scheme != "https":
        raise ValueError("Only https:// URLs are allowed")
    ctx = ssl.create_default_context()  # verifies certificate and hostname
    # Scheme is restricted to https above, so file:/ and custom schemes
    # cannot reach urlopen.
    with urllib.request.urlopen(  # nosec B310
        url, context=ctx, timeout=10
    ) as response:
        return response.read()



def import_claim_documents(archive_path):
    """Extract a customer-uploaded claim archive into ./claims_docs."""
    with tarfile.open(archive_path) as tar:
        # filter="data" (PEP 706) rejects absolute paths, ".." traversal,
        # links escaping the target, and device files.
        tar.extractall("claims_docs", filter="data")



def monthly_instalment(premium, months=12):
    """Return the instalment amount, rounded to the nearest paisa."""
    if months <= 0:
        raise ValueError("months must be at least 1")
    return round(premium / months, 2)



def is_authorised(token):
    """Return True only if token matches the configured underwriter token."""
    if not UNDERWRITER_TOKEN:
        return False  # no secret configured: nobody is authorised
    # Constant-time comparison avoids leaking the token through timing.
```

```
    return hmac.compare_digest(token.encode(), UNDERWRITER_TOKEN.encode())



def renewal_due(start, years=1):
    """Return the renewal date (YYYY-MM-DD) ``years`` after ``start``."""
    begin = datetime.strptime(start, "%Y-%m-%d").date()
    try:
        due = begin.replace(year=begin.year + years)
    except ValueError:  # 29 Feb moving to a non-leap year
        due = begin.replace(year=begin.year + years, day=28)
    return due.isoformat()



def print_quotes():
    """Print every saved quote, numbered from 1."""
    print("===== Quote Book =====")
    for i, (number, quote) in enumerate(quotes.items(), start=1):
        print(i, number, quote["name"], "Rs.", quote["premium"])



def main():
    """Run a small demonstration of the calculator."""
    p1 = calculate_premium("health", Applicant(30, False), 500000)
    p2 = calculate_premium("health", Applicant(30, False), 500000)
    p3 = calculate_premium("motor", Applicant(50, False), 800000)
    p4 = calculate_premium(
        "life", Applicant(25, False), 100000, [3000, 3000, 3000]
    )
    print("Quote 1:", p1)
    print("Quote 2 (same customer details):", p2)
    print("Non-smoker aged 50:", p3)
    print("Life with stacked discounts:", p4)
    print("Age from DOB 31-12-1990:", age_from_dob("31-12-1990"))
    print("Monthly instalment:", monthly_instalment(p3))
    print("Renewal due:", renewal_due("2025-03-15"))
    print("Imported:", import_policies(
        "<policies>"
        "<policy type='health' age='40' sum='300000'/>"
        "<policy type='motor' age='abc' sum='1'/>"
        "</policies>"
    ))
    print("Saved as:", save_quote("Kiran", p1))
    print("Authorised (wrong token):", is_authorised("wrong-token"))
    print("Server would listen on", HOST)
    print_quotes()



if __name__ == "__main__":
    main()
```